

I build systems and compiler components in C++23/C17/Rust: LLVM passes, memory-dependence analysis, CFG/SSA, graph algorithms, x86-64 code generation, and reproducible test oracles.

LLVM / C++23

Optimization pass, graph algorithms, CMake, clang-tidy, ASan/UBSan

1st of 49 · 104/104

Conservative PS-form memory-dependence analysis with an exact oracle

x86-64 pipeline

SSA-like IR, liveness, linear scan, iced-x86, and differential validation

EXPERIENCE		SKILLS
July 1 — August 31, 2026	MCST — compiler engineering intern - Implemented an LLVM LICM pass in C++ with conservative checks for side effects, memory access, loop invariance, preheaders, and speculative safety. - Built a Python harness with baseline/optimized execution, opt -verify-each, and explicit CHANGE/NO_CHANGE expectations.	C++ / toolchain C++23, C17, CMake, Linux, clang-tidy, ASan/UBSan, GitHub Actions Compiler analysis CFG, dominance, SSA, SCC, dataflow, memory dependence, optimization passes Code generation x86-64, iced-x86, liveness, linear scan, IR interpretation, isolated execution Verification exact oracles, differential/metamorphic tests, stress testing, counterexample reduction
2026	PS-form Memory Dependence Analyzer - Implemented conservative analysis of parametric memory accesses; ranked 1st of 49 with 104/104 tests. - Built an exact oracle for small domains, metamorphic tests, and reproducible WA/TL counterexamples.	EDUCATION Software Engineering student at HSE University
2026	x86-64 Codegen & Register Allocation Lab Designed SSA-like IR, liveness, linear scan, and parity checks between an IR interpreter and generated machine code.	RECOGNITION The LangDev 2026 Program Committee accepted a talk on the architecture of UniversalToolchain/Wist2 for presentation in Torremolinos, Spain. Moscow / remote
PROJECTS PS-form Memory Dependence Analyzer Ranked 1st of 49 with 104/104 tests, with reproducible wrong-answer and time-limit counterexamples. x86-64 Codegen & Register Allocation Lab A measurable pipeline with spill/reload metrics, code size, and reproducible result comparison. UniversalToolchain / Wist2 The current exact test manifest passes with zero failures; interpreter/CIL parity, clean-consumer scenarios, and reproducible compiler/runtime contracts are verified.		